

1. Tehnologii XML și Conexe:

- **Structura XML:** Înțelegerea elementelor, atributelor, rădăcinilor, ierarhiei și regulilor de bine-formatare (well-formed).
- **Validare XML:** Cunoașterea metodelor de validare a documentelor XML:
 - **DTD (Document Type Definition):** Intern (declarat în același fișier XML) și Extern (declarat într-un fișier separat).
 - **XML Schema (XSD):** Standardul recomandat pentru validare, oferind tipuri de date mai bogate și suport pentru spații de nume.
- **Procesare XML:**
 - **SAX (Simple API for XML):** Parser bazat pe evenimente, eficient pentru documente mari, dar dificil pentru navigare.
 - **DOM (Document Object Model):** API bazat pe arbori, încarcă întreg documentul în memorie, ușor de navigat și modificat, dar consumator de resurse.
 - **XSLT (Extensible Stylesheet Language Transformations):** Limbaj pentru transformarea documentelor XML în alte formate (XML, HTML, text etc.).
- **Diferența dintre limbaje de markup și meta-markup:**
 - **Limbaj de markup:** Definește o structură (ex: HTML, XHTML).
 - **Meta-markup:** Limbaj pentru definirea altor limbaje de markup (ex: XML).

2. Programare Java (Generale și Avansate):

- **Multi-threading (Concurență):**
 - Stările unui thread (New, Runnable, Running, Blocked, Waiting, Timed Waiting, Terminated).
 - Metode cheie: `start()` (care face un thread runnable și apoi running), `run()` (conține logica thread-ului), `sleep()`, `wait()`, `notify()`, `join()`.
 - Diferența între extinderea `Thread` și implementarea `Runnable`.
- **Serializare Java:**
 - Scopul serializării (persistarea obiectelor).
 - Ce se salvează și ce nu se salvează:
 1. Câmpurile non-statice (valorile lor).
 2. Câmpurile `transient` nu se salvează.
 3. Câmpurile `static` nu se salvează (fac parte din clasă, nu din obiect). La deserializare, ele păstrează valoarea curentă a câmpului static din clasă sau valoarea lor implicită (0, null, false) dacă nu sunt inițializate.
 4. Nu se salvează prototipul și corpul metodelor (acestea fac parte din bytecode-ul clasei, nu din starea obiectului).

- **Reflecție Java (`java.lang.reflect`):**

- Clasa `Class` (obținerea informațiilor despre o clasă la runtime).
- Metode precum `forName()`, `getDeclaredMethods()`, `getMethods()`, `getDeclaredFields()`, `getFields()`.
- Diferența între `getDeclaredXxx` (toate, indiferent de vizibilitate, dar doar cele declarate direct în clasă) și `getXxx` (doar cele publice, inclusiv cele moștenite).

- **Java RMI (Remote Method Invocation):**

- Conceptele de bază (client, server, registry, stub, skeleton).
- Fluxul logic al acțiunilor:
 1. **Serverul** creează obiectul remot.
 2. **Serverul** înregistrează (bind) obiectul remot în RMI Registry (cu un nume).
 3. **Clientul** face o căutare (lookup) în RMI Registry după numele obiectului.
 4. **Clientul** obține un stub (proxy) la obiectul remot.
 5. **Clientul** apelează (call) metode pe stub.
 6. **Serverul** returnează (return) rezultatul. Deci, ordinea pentru client este `lookup` -> `call`, iar pentru server este `bind`. Din perspectiva interacțiunii, `bind` trebuie să se întâmple înainte de `lookup`.

3. Dezvoltare Web (JSP/Servlets, HTTP, Cookie/Session):

- **JSP (JavaServer Pages):**

- Tag-uri standard (`jsp:useBean`).
- Atributul `scope` pentru `jsp:useBean`:
 - `page`: Obiectul este disponibil doar pentru cererea curentă și pagina curentă.
 - `request`: Obiectul este disponibil pentru cererea curentă (inclusiv pentru pagini incluse sau forwarded).
 - `session`: Obiectul este disponibil pentru întreaga durată a sesiunii utilizatorului (fiecare client are propria sesiune).
 - `application`: Obiectul este disponibil pentru toate cererile și sesiunile (un singur obiect pentru întreaga aplicație).

- **HTTP (Hypertext Transfer Protocol):**

- Protocol stateless.
- Metode (GET, POST).
- Headers (Accept, User-Agent, Host, Connection, Cookie, Set-Cookie).

- **Managementul sesiunilor în Java Servlets:**

- Obiectul `HttpSession`.
- Metoda `request.getSession(true)`: creează o sesiune nouă dacă nu există una, altfel returnează sesiunea existentă.
- Metoda `session.isNew()`: returnează `true` dacă sesiunea a fost creată pentru prima dată în timpul cererii curente.
- Mecanismul cookie-urilor (`JSESSIONID`) pentru menținerea sesiunii.
- Vizibilitatea atributelor sesiunii (specifice fiecărei sesiuni/client).

4. Rețele și Protocoale (SNMP, FTP, Base64):

- **SNMP (Simple Network Management Protocol):**

- Structura unui pachet SNMP (versiune, comunitate, PDU - `get-next-request`, `get-response`).
- **OID (Object Identifier)**: Identificatori ierarhici pentru obiectele gestionate.
- **ASN.1 DER (Abstract Syntax Notation One, Distinguished Encoding Rules)**: Standard de codificare folosit pentru a reprezenta structurile de date SNMP.
 1. Tag-uri comune: `0x30` (SEQUENCE), `0x06` (OBJECT IDENTIFIER), `0x05` (NULL), `0x02` (INTEGER), `0x04` (OCTET STRING).
 2. Decodificarea OID-urilor din formatul binar (primul octet combină primii doi noduri, apoi fiecare octet e un nod, dacă nu e extins).

- **FTP (File Transfer Protocol):**

- Porturi standard (21 pentru control, 20 pentru date în modul activ).
- Moduri de transfer de date:
 1. **Modul Activ**: Clientul trimite comanda `PORT` specificând IP-ul și portul pe care va asculta pentru conexiunea de date. Serverul inițiază conexiunea de date către client.
 2. **Modul Pasiv**: Clientul trimite comanda `PASV`. Serverul deschide un port temporar și trimite clientului IP-ul și portul pe care va asculta pentru conexiunea de date. Clientul inițiază conexiunea de date către server.
- Interpretarea comenzii `PORT` (`PORT h1,h2,h3,h4,p1,p2` unde portul este $p1 \times 256 + p2$).

- **Base64 Encoding:**

- Scopul (reprezentarea datelor binare ca text ASCII).
- Procesul de codificare:
 1. Ia 3 octeți (24 de biți) de date binare.
 2. Împarte-i în 4 grupuri de câte 6 biți.
 3. Fiecare grup de 6 biți se mapează la un caracter din tabelul Base64.

4. Folosirea caracterului de padding = dacă numărul de octeți nu este un multiplu de 3.

5. Concepte Generale de Proiectare Software:

- **Web Services: TOP-DOWN vs. BOTTOM-UP:**
 - **TOP-DOWN (Contract First):** Se începe cu definirea contractului serviciului (WSDL), apoi se generează codul și se implementează logica.
 - **BOTTOM-UP (Code First):** Se începe cu implementarea logicii serviciului în cod, apoi se generează WSDL-ul din cod.
-
-

Din [part1.pdf](#) (pare a fi legat de ASP.NET Core MVC și Securitate Web & Cloud):

1. ASP.NET Core MVC:

- **Concepte Fundamentale:**
 - **Controllers:** Rolul controlerelor în gestionarea cererilor HTTP și selectarea vizualizărilor.
 - **Actions:** Metode în cadrul controlerelor care răspund la cereri specifice.
 - **Views:** Fișiere (ex: `.cshtml` în Razor) responsabile pentru generarea HTML-ului și afișarea datelor către utilizator.
 - **Routes:** Sistemul de rutare ASP.NET care mapează URL-urile la acțiuni specifice din controlere.
 - **ViewBag:** Un mecanism pentru a transmite date de la Controller către View.
- **Crearea și Rularea Proiectelor:** Pași practici pentru a crea un proiect ASP.NET Core Web Application (Model-View-Controller) în Visual Studio.
- **Interacțiunea Controller-View:** Cum se returnează un string simplu și cum se returnează o View din acțiuni, și cum se comunică date între ele.

2. Securitate Web și Cloud (cu accent pe Microsoft Azure):

- **Concepte Generale de Securitate:**
 - **Vulnerabilități Web:**
 - **Open Redirect Attacks:** Când o aplicație permite redirectarea către un URL extern specificat de utilizator, ceea ce poate fi folosit pentru atacuri de phishing.
 - **OWASP Top 10:** Cele mai critice riscuri de securitate pentru aplicațiile web (deși nu sunt detaliate, ar trebui să le cunoști la nivel general).
 -

- **Microsoft Azure:**
 - **Azure App Service:** Un serviciu cloud pentru găzduirea aplicațiilor web, API-urilor REST și a backend-urilor mobile. Înțelege conceptele de bază ale acestui serviciu (ce oferă, la ce se folosește).
 - **Azure SQL Database:** Serviciu de baze de date relaționale în cloud. Cunoașterea modului de creare și utilizare a unei baze de date SQL în Azure.
- **Integrare cu Identitatea:** Referințe la `Identity.AspNetCore` și `Microsoft.AspNetCore.Identity.UI`, ceea ce sugerează cunoștințe despre autentificare și autorizare în aplicațiile ASP.NET Core.

Din [part2.pdf](#) (pare a fi legat de Cloud Computing, Sisteme Distribuite și Mesagerie):

1. Sisteme Distribuite și Dezvoltare de Aplicații Distribuite (DAD):

- **Definiție:** Ce este un sistem distribuit (componente interconectate, comunicare prin mesaje/migrare de procese pentru a rezolva probleme).
- **Concepte Cheie:**
 - **Comunicare și Schimb de Idei:** Subliniază importanța comunicării în DAD.

2. Cloud Computing:

- **Concepte Fundamentale de Cloud:**
 - **FaaS (Function as a Service) / Serverless Computing:** Cunoașterea conceptului de FaaS, unde dezvoltatorii scriu funcții care sunt executate la cerere, fără a gestiona serverele subiacente.
 - **AWS Lambda:** Un exemplu concret de implementare FaaS de la Amazon Web Services. Ar trebui să înțelegi cum funcționează o funcție Lambda (handler, declanșare).
- **Diferențe între Paradigme de Computing:**
 - **Fog Computing vs. Edge Computing vs. Cloud:** Înțelegerea diferențelor și cazurilor de utilizare pentru fiecare (deși nu sunt detaliate, prezența lor în material sugerează importanța conceptelor).
- **DevOps și Infrastructura Cloud:** Cunoștințe generale despre cum se integrează dezvoltarea cu operațiunile în mediile cloud.

3. Sisteme de Mesagerie (Message-Oriented Middleware - MOM):

- **JMS (Java Message Service):**
 - **Rol:** Interfața Java pentru MOM. Permite aplicațiilor Java să trimită și să primească mesaje într-un mod asincron și fiabil.
 - **Scop:** Pentru partajarea ușoară de mesaje între aplicații distribuite.

- **Implementări:** Exemplul menționat este Apache TomEE (un server Java Web & Application cu implementare JMS).
- **Apache Kafka:** Un sistem de streaming distribuit, folosit pentru construirea pipeline-urilor de date în timp real și a aplicațiilor de streaming.

Recomandări pentru Pregătire:

- **Revizuieste Slide-urile:** Parcurge în detaliu slide-urile, deoarece ele conțin informația esențială.
- **Aprofundează Conceptele:** Pentru fiecare punct din sinteza de mai sus, asigură-te că înțelegi nu doar definiția, ci și cum funcționează conceptul în practică.
- **Example Practic:** Dacă există secțiuni cu exemple de cod (cum e cazul ASP.NET Core), înțelege logica și sintaxa.
- **Terminologie:** Fii familiarizat cu terminologia specifică fiecărui domeniu (ex: "stateless", "handler", "stub", "skeleton", "PDU", "OID" etc.).
- **Relațiile dintre Concepte:** Încearcă să vezi cum se leagă diferitele concepte între ele (ex: cum sunt folosite cookie-urile pentru sesiuni în HTTP, sau cum JMS e o implementare a MOM).